

IFAC PapersOnLine 56-2 (2023) 3164–3169

IMPACT: Uma cadeia de ferramentas para modelo não linear **IMPACT:** Uma cadeia de ferramentas para modelo não linear **IMPACT:** Uma cadeia de ferramentas para modelo não linear

[illegible][illegible][illegible]

Spinal Health e a Spinal Cord Injury Society of the United States para a criação de uma solução baseada em IA, demonstrando a eficácia e

Balavras-chave: Controle Preditivo de Modelo; Cadeia de Ferramentas; Prototipagem; Balavras-chave: Controle Preditivo de Modelo; Cadeia de Ferramentas de Software; Prototipagem; Implantação Palavras-chave: Controle Preditivo de Modelo; Cadeia de Ferramentas de Software; Prototipagem; Implantação Palavras-chave: Controle Preditivo de Modelo; Cadeia de Ferramentas de Software; Prototipagem; Implantação

[illegible][illegible]

Introdução: O SPC é um conjunto de ferramentas para a solução de problemas de engenharia (EN).
 Metodologia: Os estudos são conduzidos de forma paralelizada, oferecendo um vasto leque de soluções para cada problema (na maioria das vezes, mais de uma).
 Resultados: Os resultados são apresentados de forma paralelizada, oferecendo um vasto leque de soluções para cada problema (na maioria das vezes, mais de uma).
 Conclusão: O SPC é um conjunto de ferramentas para a solução de problemas de engenharia (EN).
 Palavras-chave: Engenharia, Solução de Problemas, Ferramentas, Engenharia, Solução de Problemas, Ferramentas, Engenharia, Solução de Problemas, Ferramentas.

Na literatura, a combinação de métodos ECR de dados de vendas, N-1 e do GP, se aplicáveis, pode proporcionar uma visão mais abrangente e detalhada da realidade econômica e financeira da empresa, permitindo uma análise mais completa e integrada dos dados disponíveis. Essa abordagem integrada pode contribuir para uma melhor compreensão da situação financeira da empresa, auxiliando na tomada de decisões estratégicas e operacionais. No entanto, é importante ressaltar que a aplicação desses métodos deve ser feita de forma cuidadosa, considerando as particularidades de cada empresa e o contexto econômico em que ela atua. Além disso, a integração dos dados deve ser feita de forma transparente e ética, respeitando a privacidade dos dados e a legislação aplicável.

Make é o centro de pesquisa estratégica flamengo para a indústria de manufatura.

Nota Final: Este trabalho foi realizado em parceria com a empresa produtora de papel e celulose, a **Aracruz Celulose**, e a **Aracruz Papel e Celulose**, e a **Aracruz Celulose**, e a **Aracruz Papel e Celulose**.

Agradecemos a todos os participantes que contribuíram para este trabalho, indústria de manufatura e serviços, e em especial, a todos os colaboradores da indústria de manufatura.

² Consultez https://github.com/meco-group/dynamic_optimization_Control. A Flanders

2 Consulte https://github.com/meco-group/dynamic_optimization_Control. A Flanders
2 Consulte https://github.com/meco-group/dynamic_optimization_Control. A Flanders

²eco-dynamics optimization (Flor et al.) "A fluid dynamics framework for gradient descent in a structured domain"

1 A. Florez e A. Astudillo contribuíram igualmente para este trabalho. 2 a indústria manufatureira

igualmente para este trabalho. Copyright © 2023 Os Autores. Este é um artigo de acesso aberto sob a

Revisão por pares sob responsabilidade da Federação Internacional de Controle Automático.

A modelagem de alto nível significa que o usuário fica sobrecarregado com Escrita em formas canônicas e fornecimento de derivados.

Para abordar as questões mencionadas, apresentamos o IMPACT. uma cadeia de ferramentas de código aberto que visa facilitar a fluxo de trabalho de especificação, prototipagem e implantação de NMPC. Oferece integração vertical, ao mesmo tempo que proporciona modularidade, com sobrecarga mínima em tempo de execução. Principais características Os aspectos do IMPACT são descritos da seguinte forma:

- Ela fornece uma interface de programação de aplicativos (API) unificada de alto nível para modelar OCPs;
- É multilíngue (escrito em Python e gerenciada automaticamente³ vínculos MATLAB) e plataforma por cross-vides;
- Oferece uma gama de métodos de transcrição direta que são emparelhados com solucionadores QP/NLP de última geração por meio de Plugins CasADi como HPIPM (Frison e Diehl, 2020), PROX-QP (Bambade et al., 2022), IPOPT (Wächter e Biegler, 2006), FSLP (Kiessling et al., 2022);
- Oferece alternativas à transcrição direta integrada. ao interligar solucionadores/frameworks OCP via Rockit plugins – no momento Acados, GRAMPC e FATROP (Vanroye et al., 2022);
- Exporta bibliotecas MPC específicas para cada problema com uma API C unificada, útil para prototipagem rápida e implantação em ambientes simulados e reais;
- Ele exporta um bloco Simulink específico para o problema, tornando-o possível. Utilização da biblioteca MPC exportada.

O IMPACT é distribuído sob a licença LGPLv3 e seus respectivos códigos.

O código-fonte está disponível em <https://gitlab.kuleuven.be/meco-software/impact>. É construído sobre as poderosas ferramentas de otimização e controle ótimo CasADi. (Andersson et al., 2019) e Rockit (Gillis et al., 2020).

O restante deste artigo está organizado da seguinte forma. A Seção 2 apresenta conceitos preliminares sobre a formulação. e transcrição do OCP que sustenta o MPC. Em Seção 3, detalhes sobre a estrutura geral e o fluxo de trabalho de São apresentados os resultados do IMPACT. Um exemplo de aplicação usando o IMPACT. é descrito na Seção 4. Finalmente, encerramos o artigo com Conclusões e observações sobre trabalhos futuros.

2. PROBLEMA DE CONTROLE ÓTIMO

Para um horizonte temporal $t \in [t_0, t_f]$, vetor de estado do sistema $x(t) \in \mathbb{R}^{n_x}$, vetor de entrada $u(t) \in \mathbb{R}^{n_u}$, vetor de estado algébrico $z(t) \in \mathbb{R}^{n_z}$ em $\mathbb{R}^{n_z}$ e parâmetros $p \in \mathbb{R}^{n_p}$, Considere formulações OCP gerais da forma canônica.

$$\begin{aligned} & \underset{x, u, z}{\text{minimizar}} \quad \int_{t_0}^{t_f} V(x(t), u(t), p, t) dt + V_f(x(t_f), p) \quad (1a) \\ & \text{sujeito a } B(x(t_0), p) \leq 0, \quad (1b) \\ & \quad \dot{x}(t) = \bar{V}(x(t), u(t), z(t), p), \quad t \in [t_0, t_f], \quad (1c) \\ & \quad \bar{V}(x(t), u(t), z(t), p) = 0, \quad t \in [t_0, t_f], \quad (1d) \\ & \quad h(x(t), u(t), p) \leq 0, \quad t \in [t_0, t_f], \quad (1e) \end{aligned}$$

onde $V(\cdot)$ e $V_f(\cdot)$ em (1a) são funções não lineares suaves que definem os termos de Lagrange e Mayer, respectivamente, p é um vetor de parâmetros que normalmente contém um vetor de medições ou estimativas de estado $x_{meas} \in \mathbb{R}^{n_x}$, (1b) define uma restrição de fronteira que normalmente assume a forma de

$x(t_0) = x_{meas}$, (1c) e (1d) definem um sistema de equações diferenciais algébricas (EDA) que representa o modelo do sistema, enquanto (1e) representam restrições gerais de caminho. Observe que A inclusão do vetor de estado algébrico z e da equação algébrica (1d) não é necessária e pode ser omitida para Sistemas representados exclusivamente por equações diferenciais ordinárias (EDO). O OCP especificado pelo usuário no IMPACT. (ver Seção 3.3) é automaticamente transformado em forma canônica (1).

A transcrição direta de (1) leva a uma dimensão finita problema de otimização da forma

$$\min_{w} f(w) \text{ s.t. } g(w) = 0, h(w) \leq 0, \quad (2)$$

onde $w \in \mathbb{R}^{n_w}$ é o vetor de variáveis de decisão, definido ao considerar $N := T / \Delta t \in \mathbb{N}$ pontos de discretização dentro do horizonte de previsão $T := (t_f - t_0) / \Delta t \in \mathbb{R}_{>0}$ e um tempo de amostragem $\Delta t \in \mathbb{R}_{>0}$. A primeira e a segunda ordem as derivadas que surgem nas condições de otimalidade de (2) são geralmente grandes e esparsos.

3. DESCRIÇÃO DA CADEIA DE FERRAMENTAS

Esta seção apresenta uma visão geral da estrutura do A cadeia de ferramentas IMPACT. Primeiramente, descreve as dependências fundamentais da cadeia de ferramentas e, em seguida, apresenta... os elementos da cadeia de ferramentas e um fluxo de trabalho do MPC da especificação à implantação. Finalmente, detalha o uso de A API unificada para comunicação com os solucionadores MPC.

3.1 Dependências Fundamentais

As dependências de software fundamentais necessárias para instalar e usar o IMPACT são: CasADi, Rockit e o Módulos Python pyyaml e lxml.

O IMPACT depende do CasADi para lidar com expressões que modelar o objetivo e as restrições como grafos de expressão, e para permitir o salvamento/carregamento de funções e algoritmos e sua transferência entre Python e MATLAB.

Se instruído pelo usuário, o IMPACT utiliza o Rockit para executar um método de transcrição OCP escolhido. No momento, disparo múltiplo, disparo único, colocação direta e São suportados métodos B-spline. As derivadas do objetivo e das restrições do NLP resultante são obtidas através de Diferenciação algorítmica do CasADi que explora a esparsidade (AD), opcionalmente tornado mais eficiente pela geração de código C.

O Rockit também fornece interfaces para OCP de terceiros. frameworks como GRAMPC, Acados e FATROP. Inicialmente concebidas para uso offline, essas interfaces estão sendo Refatorado para usar a API C de geração de código CasADi, de forma que Eles podem ser incorporados em expressões CasADi e são compatível com geração de código C. Por fim, o Rockit depende no CasADi para importar unidades de protótipo funcionais (FMU) para representações de entrada/saída de sistemas.

As bibliotecas pyyaml e lxml são usadas no IMPACT. para lidar com arquivos YAML e XML necessários no processo de carregando modelos de sistemas e gerando blocos Simulink ao exportar a solução, respectivamente.

3.2 Estrutura da cadeia de ferramentas IMPACT

Uma ilustração gráfica da estrutura geral do A cadeia de ferramentas IMPACT é mostrada na Figura 1. CasADi e Rockit

³ Ver https://gitlab.kuleuven.be/meco-software/python_matlab

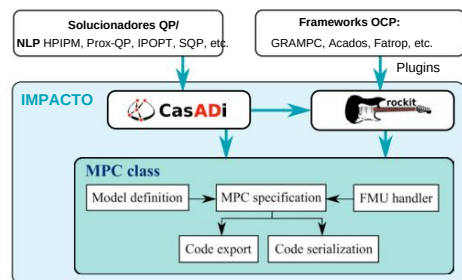


Figura 1. Visão geral da arquitetura da cadeia de ferramentas IMPACT, mostrando a interação entre os módulos.

Desempenham um papel vital no IMPACT, fornecendo funcionalidades essenciais à cadeia de ferramentas, como, por exemplo, manipulação de expressões e interface com solvers de terceiros. De fato, a dependência do IMPACT em relação ao CasADi e ao Rockit, e a modularidade inerente a essas dependências, permitem a disponibilidade direta e transparente de novos solvers no IMPACT sempre que uma interface para esses solvers for adicionada ao CasADi ou ao Rockit.

Além do CasADi e do Rockit, o IMPACT é composto pela classe MPC, que é instanciada pelo usuário para utilizar todas as funcionalidades do IMPACT. Essa classe é dividida em cinco submódulos, que são descritos a seguir.

1) Definição do modelo: O IMPACT fornece ao usuário duas maneiras de definir modelos de sistemas (não lineares) para serem usados na definição do OCP: (i) definindo manualmente variáveis de estado e controle, além de equações diferenciais e algébricas, como feito no Rockit ou (ii) usando arquivos YAML – ou seja, arquivos legíveis por humanos para serialização de dados – usando o método `add_model()` da classe MPC.

Ao utilizar arquivos YAML, nosso objetivo é padronizar a definição de modelos baseados em EDOs ou EDAs para uso em qualquer ferramenta de software compatível com YAML. Essa representação de modelo permite o uso tanto da definição inline de equações quanto de funções CasADi serializadas externamente, ou seja, arquivos `*.casadi`, como mostrado nos dois trechos de código YAML de um sistema de pêndulo a seguir.

```

equações:
  embutidas:
    ode:
      phi: dphi dphi:
        L*cos(phi)*sin(phi) ...
    estados_diferenciais: - nome: phi -
      nome: dphi
  controles:
    - nome: F
  constantes:
    em linha:
      L: 2

```

```

equações:
  externas:
    tipo: casadi_serializado nome_do_arquivo:
      ode.casadi estados_diferenciais:
        - nome: phi - nome:
          dphi controles:
            - nome: F

```

2) Manipulador de FMU: Este módulo permite ao usuário carregar FMUs – um padrão da indústria para definir contêineres que representam modelos de caixa-preta (entrada/saída) – como funções CasADi para serem usadas na definição do OCP. Ao chamar o método `add_simulink_fmu()` da classe MPC, a FMU é encapsulada em um arquivo C++, que é então compilado e carregado de volta como uma função externa CasADi. Quando a FMU fornece derivadas (derivadas progressivas de acordo com o padrão FMI 2.0), estas podem ser usadas. Caso contrário, o CasADi usa diferenças finitas. Em um estágio posterior, o manuseio de FMUs deverá ser incorporado à abordagem de arquivo YAML.

3) Especificação MPC: Este módulo herda a funcionalidade da classe OCP do Rockit para permitir a especificação dos OCPs que sustentam o MPC. Com o OCP

A classe Rockit permite a definição de OCPs (multiestágios) por meio de: (i) gerenciamento de variáveis simbólicas para estados do sistema x , entradas do sistema u e estados algébricos z ; (ii) associação de dinâmicas a x e equações algébricas a z , como em (1c) e (1d); (iii) composição da função objetivo (1a) pela adição de funções avaliadas em diferentes instantes dentro do horizonte do OCP; e (iv) definição de restrições de caminho ou de fronteira para expressões ou variáveis, como em (1b) e (1e). Ao escolher um plugin OCP de terceiros, podem ser aplicadas restrições à especificação do problema. A especificação MPC também é usada para recursos de pós-processamento de resultados que permitem ao usuário, por exemplo, recuperar os valores de variáveis ou expressões específicas na solução do OCP ou interpolar a solução em uma grade refinada de pontos de integração.

4) Serialização de código: Este módulo permite a serialização do grafo de expressão criado pelo CasADi para definir o solucionador MPC. Isso possibilita salvar uma instanciamento do solucionador MPC (incluindo as funções que definem o OCP) em um arquivo que pode ser carregado a partir de Python, MATLAB ou C++.

Este módulo depende dos métodos `save()` e `load()` da classe MPC. De maneira semelhante, instâncias do módulo Python IMPACT também podem ser salvas e carregadas.

5) Exportação de código: Ao chamar o método `export()` da classe MPC, o usuário gera múltiplos artefatos que permitem a prototipagem e a implantação de um solucionador dentro de uma implementação MPC. Os principais artefatos representados na Figura 2 são: (i) uma biblioteca IMPACT acessível através da API C do IMPACT; (ii) uma biblioteca de backend numérico contendo uma formulação concreta de baixo nível do problema, acoplada a um solucionador que pode ser incluído como código C totalmente independente, vinculado com uma dependência da biblioteca de tempo de execução C++ CasADi e um plugin de solucionador escolhido, como o IPOPT, ou vinculado como dependências de terceiros, como OSQP ou GRAMPC; e (iii) um bloco IMPACT Simulink pronto para MPC.

A biblioteca IMPACT é acompanhada por interfaces em Python. Mais detalhes podem ser encontrados na Seção 3.4.

O código C independente não possui dependências e requer apenas um compilador C para ser implementado em qualquer sistema compatível. Ele exige que cada elemento da instanciamento do solucionador MPC seja compatível com a geração de código – por exemplo, se o usuário selecionar IPOPT, que não pode ser gerado por código, nenhum código C independente do solucionador MPC poderá ser gerado.

O bloco IMPACT do Simulink é baseado em uma função S personalizada em C, gerada por código. O IMPACT gera um contêiner `.slx` baseado na API C e vinculado aos arquivos C gerados, podendo ser usado diretamente em um modelo do Simulink.

O Simulink tornou-se uma ferramenta padrão para simulação e prototipagem de sistemas de controle. Este artefato permite que os usuários do IMPACT criem protótipos de seus MPCs projetados com facilidade, sem etapas adicionais. Além disso, se o arquivo C vinculado for autocontido, o bloco Simulink gerado será compatível com o Simulink Coder. Dessa forma, o bloco Simulink do IMPACT pronto para MPC pode ser gerado como parte de um modelo Simulink maior e implementado em sistemas de tempo real sem esforço adicional de engenharia.

A compatibilidade com o Simulink Coder também é esperada quando as dependências de terceiros são de código aberto e baseadas em C, mas esse fluxo de trabalho ainda não foi validado.

Os três artefatos incluem funções CasADi para o solucionador MPC (com entradas paramétricas e de inicialização a quente), funções forward

simulação da dinâmica do sistema, DAEs, objetivo de desempenho, entre outros. Além disso, fornecem estatísticas da solução, como tempo de solução, status, otimalidade da solução e número de iterações. Isso oferece ao usuário grande flexibilidade para implementar e prototipar a solução MPC em um laço de controle simulado ou real. Observe que, uma vez gerados os artefatos, os únicos valores que podem ser modificados no problema são os parâmetros p .

3.3 Fluxo de trabalho da cadeia de ferramentas

Esta seção detalhou a arquitetura do IMPACT e suas dependências. Agora, é necessário explicar o fluxo de trabalho proposto pelo IMPACT para que o usuário especifique um MPC, crie um protótipo de sua solução e a implemente.

Uma visão geral do fluxo de trabalho é apresentada na Figura 2.

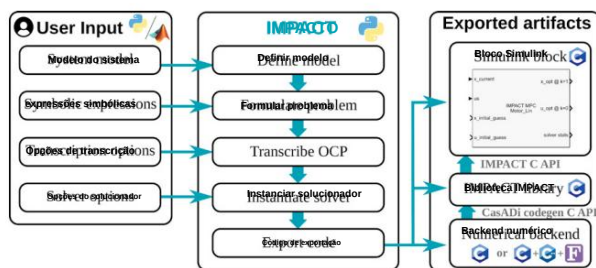


Figura 2. Visão geral do fluxo de trabalho da cadeia de ferramentas IMPACT.

O fluxo de trabalho na caixa de ferramentas IMPACT é descrito a seguir por meio de um exemplo simples envolvendo o controle de um pêndulo. Uma descrição de cada etapa é fornecida, seguida pelo trecho de código associado a ela.

Primeiramente, o usuário deve importar o módulo IMPACT e instanciar um objeto da classe MPC. O argumento $T=2.0$ define o horizonte de previsão T para um valor fixo. Entretanto, T pode ser considerado como uma variável de decisão dentro de um problema de tempo livre passando $T=FreeTime(2.0)$ como argumento, onde 2.0 é um palpite inicial para esta variável.

```
from impact import * mpc
= MPC(T=2.0)
```

Em seguida, o modelo do sistema deve ser declarado. Como mencionado na Seção 3.2, isso pode ser feito escrevendo manualmente a EDO ou EAD – ou seja, usando instâncias de estados e entradas dentro da classe MPC – ou, como mostrado no trecho de código abaixo, carregando um arquivo YAML – como no exemplo de um sistema de pêndulo apresentado na Seção 3.2. `pendulum` =

```
mpc.add_model("pendulum", "pendulum.yaml")
```

Uma vez definido o modelo, o usuário pode instanciar os elementos do vetor de parâmetros p , como o estado atual (x_0), um estado final desejado (x_f) e os pesos (Wt) a serem usados no objetivo.

```
x_0 = mpc.parameter("x_0", pendulum.nx) x_f =
mpc.parameter("x_f", pendulum.nx)
Wt = mpc.parameter("Wt", 2)
```

Em linhas gerais, o objetivo e as restrições são definidos usando os métodos `mpc.add_objective()` e `mpc.subject_to()` diretamente, com nomes de estado e controle.

Note que, ao usar `mpc.at_t0()` ou `mpc.at_tf()`, o usuário pode especificar restrições de limite.

```
# Objetivo
mpc.add_objective(mpc.integral(Wt[0]*pendulum.F**2 + Wt[1]*pendulum.dphi**2))
# Restrições de contorno
mpc.subject_to(mpc.at_t0(pendulum.x) == x_0)
```

```
mpc.subject_to(mpc.at_tf(pendulum.x) == x_f)
# Restrições de caminho
mpc.subject_to(-2 <= (pendulum.F <= 2))
```

O solucionador de otimização não linear a ser utilizado é definido por meio do método `mpc.solver()`, para o qual o usuário fornece diversas opções dependendo do solucionador selecionado.

Neste caso, foi selecionado o método SQP do CasADi.

```
mpc.solver("sqpmethod", options={...})
```

O método de transcrição é definido usando um método direto fornecido pelo Rockit ou um método externo fornecido por um plugin para uma ferramenta de otimização como o GRAMPC. Aqui, N corresponde ao número de pontos de discretização N , enquanto `intg='rk'` define um integrador de Runge-Kutta de 4ª ordem para discretizar a dinâmica do sistema.

```
# Método direto (MultipleShooting, SingleShooting, etc.), opção 1 método =
MultipleShooting(N=40, intg='rk')
# Método externo ('grampc', 'fatrop', etc.), opção 2 método =
external_method('grampc', N=40, grampc_options=...)
# Defina o método selecionado no ambiente MPC mpc.method(method)
```

O usuário agora pode definir os valores dos parâmetros p e executar o solucionador diretamente em Python.

```
mpc.set_value(x_0, [0.5, 0.0, 0.0]) # Define os parâmetros
mpc.set_value(x_f, [0.0, 0.0, 0.0])
mpc.set_value(Wt, [1, 1]) solution
= mpc.solve() # Resolve o OCP
```

Finalmente, o usuário pode gerar os três artefatos explicados na Seção 3.2 executando o seguinte código.

```
mpc.export("pendulo")
```

Para prototipagem rápida do código exportado, o usuário pode: (i) executar os exemplos "Olá, mundo!" gerados em C ou Python, que usam a API C, ou (ii) usar o bloco Simulink gerado a partir do arquivo .slx gerado em um modelo Simulink.

Como estamos usando o método SQP do CasADi, que é compatível com a geração de código, neste exemplo poderíamos implantar o código gerado ou o bloco Simulink em uma arquitetura de destino em tempo real.

3.4 API C de IMPACTO

A API C do IMPACT define uma API unificada para usar o solucionador MPC exportado pelo IMPACT. Ela representa o artefato central da funcionalidade de exportação de código e é uma camada de abstração que torna a implementação agnóstica ao conteúdo do solucionador MPC gerado, por exemplo, código C autocontido com solucionador C, código C++ que ainda depende do CasADi e/ou de solucionadores de terceiros. Ela fornece funções abrangentes para definir as entradas do solucionador MPC (parâmetros) e obter os valores das variáveis da solução otimizada. Para fins de completude, apresentamos abaixo um exemplo mínimo do uso da API C do IMPACT para o exemplo do pêndulo apresentado na Seção 3.3.

```
impact_struct* m = impact_initialize(); impact_set(m,
"x_0", IMPACT_ALL, 0, x_meas, IMPACT_FULL); // Define um parâmetro impact_solve(m); // Resolve
um único problema de controle ótimo (OCP)
// Obter entrada ideal para a primeira instância impact_get(m,
"u_opt", IMPACT_ALL, 0, u, IMPACT_FULL);
```

Aqui, `impact_initialize()` instancia a biblioteca IMPACT exportada e retorna um ponteiro `m` para ela. Usando o método `impact_set()`, o parâmetro `'x_0'` é definido com um valor definido pelo usuário, `'x_meas'`. O terceiro argumento, `'IMPACT_ALL'`, informa ao IMPACT que todos os elementos do parâmetro especificado estão definidos. O usuário pode usar

Em vez de IMPACT_ALL, use "pendulum.phi" para definir o parâmetro correspondente ao ângulo phi, por exemplo. O quarto argumento, 0, define a instância de tempo dentro do horizonte para a qual o parâmetro é definido. Outra opção é IMPACT_EVERYWHERE, que define o parâmetro para todo o horizonte. O sexto argumento combina flags que representam repetição (IMPACT_FULL, IMPACT_HREP) e ordenação de dados (IMPACT_ROW_MINOR, IMPACT_COLUMN_MAJOR). A função impact_solve() executa o solucionador MPC. Em seguida, um elemento da solução – ou seja, a entrada de controle ótima u_{opt} – é recuperado e atribuído a uma variável u usando impact_get(), com os argumentos espelhando impact_set(). Este exemplo mostra a solução de um OCP. Para implementar (N)MPC, os métodos impact_set(), impact_solve() e impact_get() são executados em um loop. Mais métodos são definidos na API, por exemplo, para recuperar estatísticas e realizar depuração.

4. EXEMPLO DE APLICAÇÃO USANDO IMPACTO

Esta seção demonstra o uso e a implementação do MPC utilizando o IMPACT.

Controlamos o movimento ponto a ponto (posição angular) de um motor CC utilizando uma máquina alvo em tempo real Speedgoat SN7233. O esquema de controle estima uma perturbação constante que representa dinâmicas não modeladas – por exemplo, perturbações de entrada e atrito estático – contrabalançando seu efeito e alcançando um posicionamento sem desvios.

4.1 Descrição do Sistema

O hardware possui dois componentes: o motor CC e a máquina alvo em tempo real que executa o algoritmo MPC e faz a interface com o driver do motor e um codificador rotativo.

O eixo do motor CC está conectado a uma carga e ao codificador rotativo, que fornece uma medição y da posição angular do rotor $\hat{y}$ e $\dot{y}$ R. O motor possui um driver que recebe uma tensão analógica v_{motor} e $\dot{y}$ [10, 10] do conversor digital-analógico (DAC) do Speedgoat. $\hat{y}$ R e uma entrada de controle

Para um vetor de estado $x := [\hat{y}, \dot{y}]^T$ $u := v_{motor}$, um modelo linear de segunda ordem para o motor $\ddot{x} = Ax + Bu$ (3)

foi identificado experimentalmente usando o

LCtoolbox (Verbandt et al., 2018). Este modelo contém um integrador e um polo real que modela o atrito do motor linear, mas não considera o componente de atrito estático.

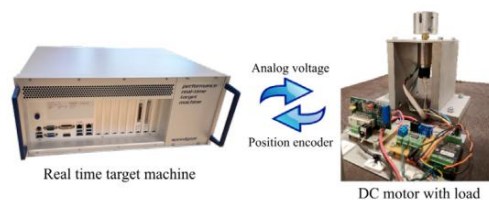


Figura 3. Visão geral do sistema.

O Speedgoat SN7233 é uma máquina alvo em tempo real, programada via Simulink, que permite a implementação de controladores em, por exemplo, CPUs multi-core, FPGAs e PLCs, e facilita a prototipagem rápida de controladores e simulações hardware-in-the-loop por meio de uma integração perfeita com o Simulink Real-Time. O alvo executa o algoritmo MPC – baseado em um bloco IMPACT do Simulink –, aplica uma tensão analógica v_{motor} ao motor e lê os sinais digitais do encoder rotativo, conforme ilustrado na Figura 3.

4.2 Definição do Problema

Para alcançar um posicionamento preciso do rotor, ou seja, contrabalançando o efeito da discrepância entre o modelo e a planta e uma perturbação de entrada constante d e $\dot{y}$ R, definimos um MPC livre de offset (Rawlings et al., 2017, Seção 1.5). Além da solução do OCP (1), esta metodologia envolve (i) aumentar a dinâmica do sistema dentro do algoritmo de estimação com a dinâmica de d e (ii) resolver um problema de otimização – o problema de estado estacionário ou seletor de alvo – que define referências para $\{x, u\}$ no OCP (1) com base nos estados estimados e na perturbação, para compensar os efeitos da perturbação enquanto rastreia uma referência.

A estrutura MPC sem offset é sintetizada na Fig. 4.

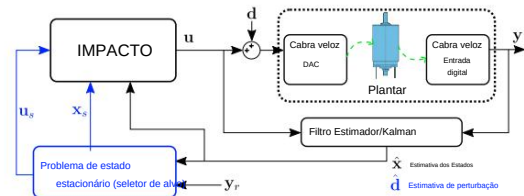


Figura 4. Diagrama de uma implementação MPC sem offset, com diferenças em relação ao MPC tradicional em azul e conexões físicas com o motor em verde.

Antes de apresentar detalhes sobre a definição de OCP (1) para esta aplicação, vamos descrever o estimador e o problema de estado estacionário. O estimador – ou seja, um filtro de Kalman – fornece uma estimativa $\hat{x}^k$ de um vetor de estado aumentado $\hat{x}^k := [x^T, d^T]^T$ cuja dinâmica é baseada em uma representação $\hat{x}^k$ discretizada de (3) e na suposição de uma perturbação constante. O problema de estado estacionário corresponde a um problema de otimização que considera o ano de referência y_r e a perturbação estimada $\hat{d}$ para definir os pontos de ajuste $\{x_s, u_s\}$ para $\{x, u\}$ em OCP (1). Para este exemplo, a solução de tal problema pode ser encontrada analiticamente como a solução de um sistema de equações lineares. O leitor pode consultar Rawlings et al. (2017) para obter mais detalhes sobre essa formulação.

Em vez de minimizar o erro de saída $y - \hat{y}_r$, o OCP (1) visa minimizar o desvio entre os pontos de ajuste $\{x_s, u_s\}$ e as variáveis do sistema $\{x, u\}$. Portanto, o objetivo (1a) é definido por $V(\cdot) := x(t_f)^T \hat{y}_s + u(t_f)^T \hat{u}_s + V_f(\cdot) := x(t_f)^T \hat{y}_s + x_s^T$

onde Q, R, Q_f são matrizes de peso. Na restrição de contorno (1b), $x_{meas} = \hat{x}$, ou seja, excluindo a perturbação estimada. A dinâmica (1c) é descrita pela EDO (3). Como o sistema não apresenta equações algébricas, (1d) é omitido. As restrições de contorno (1e) são dadas pelos limites inferior e superior em v_{motor} , de forma que $\hat{y}_{10} \leq u(t) \leq \hat{y}_{10} + \hat{y}_t$ $[t_0, t_f]$.

4.3 Detalhes da Implementação

Para implementar o bloco IMPACT Simulink que resolve OCP (1), seguimos o fluxo de trabalho descrito na Seção 3.3.

Usamos $N = 50$, $\hat{y}_t = 3,33$ [ms] e $T = 166,66$ [ms].

O modelo (3) é definido em um arquivo YAML e carregado no objeto MPC. Cinco parâmetros são definidos, a saber, x_{meas} , x_s , u_s , Q , R e Q_f . O objetivo e as restrições do OCP (1) são definidos como na Seção 4.2. Utilizamos o solucionador QP baseado em conjuntos ativos QRQP (Andersson e Rawlings, 2018) no CasADi e disparo múltiplo como método de transcrição.

com um integrador Runge-Kutta de 4ª ordem para discretização do sistema. O solucionador MPC é exportado para gerar o bloco Simulink. Este bloco é carregado em um modelo Simulink onde o estimador e o problema de estado estacionário são implementados, e onde os blocos de entrada/saída do Speedgoat são carregados. O bloco IMPACT é conectado aos outros blocos como na Fig. 4 para fechar o laço. A referência é $y(t) = \dot{y}3$ [rad], $\dot{y}t$ [1, 3], e zero caso contrário. Enquanto $d = 0,5$ [V] $\dot{y}t$ 8.

4.4 Resultados

Uma vez configurado o modelo Simulink, chamamos a operação de execução em tempo real do Simulink Real-Time, que gera o código usando o Simulink Coder e o executa no Speedgoat. Os resultados da implementação MPC sem offset são comparados com uma implementação MPC tradicional, ou seja, sem estimativa de perturbação e execução do problema em regime permanente. A Figura 5 mostra a evolução da posição angular $\dot{y}$ e da tensão de entrada u com ambas as implementações. Aqui, o MPC sem offset consegue seguir a referência com offset zero, enquanto o MPC tradicional se desvia da referência antes (devido aos efeitos da incompatibilidade entre modelo e planta) e depois da aplicação da perturbação em $t = 8$.

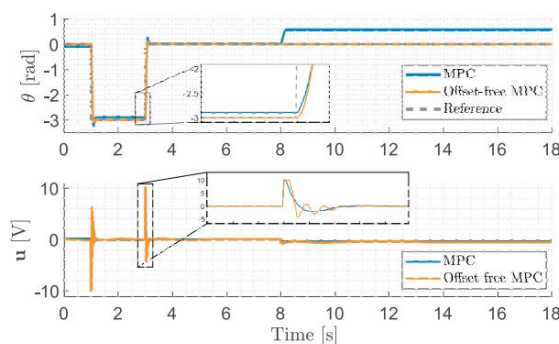


Figura 5. Evolução da posição angular $\dot{y}$ e da tensão de entrada u durante a execução da aplicação.

O IMPACT facilitou a especificação, os testes e a implementação de MPC, permitindo a prototipagem rápida do problema e o teste de diversos solvers usando os artefatos gerados. Além disso, o bloco IMPACT para Simulink pode ser facilmente integrado a modelos maiores para simulação ou implementação em hardware real. Embora este exemplo apresente um sistema linear, o IMPACT permite a definição de sistemas representados por equações lineares ou não lineares. Outras aplicações testadas com o IMPACT incluem a implementação de NMPC em um alvo Speedgoat e em um PC embarcado Beckhoff TwinCAT para uma aplicação de movimento ponto a ponto com um robô SCARA paralelo. No futuro, o IMPACT será testado em uma placa controladora DSpace e estendido para uso em FPGAs e GPUs.

5. CONCLUSÕES

Neste artigo, apresentamos o IMPACT, um conjunto de ferramentas para controle preditivo de modelos não lineares que reduz a complexidade de engenharia das diferentes etapas, desde a especificação do problema até a implementação da solução MPC. Os diferentes módulos que compõem o conjunto de ferramentas, suas dependências e seu fluxo de trabalho de uso foram descritos. No vasto campo do software acadêmico, o IMPACT se destaca por oferecer uma abordagem unificada.

Uma forma multilíngue de alto nível para modelar OCPs e uma API C unificada para comunicação com controladores MPC exportados. As direções futuras incluem a implementação de um método que permita a exportação da API C como uma FMU. Também validaremos o conjunto de ferramentas em cenários mais complexos envolvendo sistemas robóticos complexos e carros autônomos.

REFERÊNCIAS

- Andersson, JA, Gillis, J., Horn, G., Rawlings, JB e Diehl, M. (2019). Casadi: uma estrutura de software para otimização não linear e controle ótimo. *Mathematical Programming Computation*, 11(1), 1–36.
- Andersson, JA e Rawlings, JB (2018). Análise de sensibilidade para programação não linear em CasADi. *IFAC-PapersOnLine*, 51(20), 331–336.
- Bambade, A., El-Kazdadi, S., Taylor, A., e Carpentier, J. (2022). PROX-QP: Mais um solucionador de programação quadrática para robótica e outras áreas. Em *RSS 2022 - Robótica: Ciência e Sistemas*. Nova York, Estados Unidos.
- Chen, Y., Bruschetta, M., Picotti, E. e Beghi, A. (2019). MATMPC - uma caixa de ferramentas baseada em MATLAB para controle preditivo de modelos não lineares em tempo real. Apresentado na 18ª Conferência Europeia de Controle (ECC) de 2019. IEEE.
- Deng, H. e Ohtsuka, T. (2018). Um conjunto de ferramentas de geração de código paralelo para controle preditivo de modelo não linear. Em *Anais da 57ª Conferência IEEE sobre Decisão e Controle*, 4920–4926. Miami, EUA.
- Englert, T., V'olz, A., Mesmer, F., Rhein, S., e Graichen, K. (2019). Uma estrutura de software para controle preditivo de modelo não linear incorporado usando uma abordagem lagrangiana aumentada baseada em gradiente (GRAMPC). *Otimização e Engenharia*, 20(3), 769–809.
- Frison, G. e Diehl, M. (2020). Hpipm: uma estrutura de programação quadrática de alto desempenho para controle preditivo de modelo. doi:10.48550/arxiv.2003.02547.
- Gillis, J., Vandewal, B., Pipeleers, G., e Swevers, J. (2020). Modelagem sem esforço de problemas de controle ótimo com rokit. Em 39º Encontro Benelux sobre Sistemas e Controle, Data: 10/03/2020 - 12/03/2020, Local: Elspeet, Holanda.
- Katayama, S. e Ohtsuka, T. (2020). Ferramenta de geração automática de código para controle preditivo de modelo não linear com jupyter. *IFAC-PapersOnLine*, 53(2), 7033–7040.
- Kiessling, D., Zanelli, A., Nurkanović, A., Gillis, J., Diehl, M., Zeilinger, M., Pipeleers, G., e Swevers, J. (2022). Um algoritmo viável de programação linear sequencial com aplicação a problemas de planejamento de trajetória com tempo ótimo. In *Anais da 61ª Conferência IEEE sobre Controle e Decisão*. Cancún, México.
- Mathworks (2019). Model Predictive Control Toolbox. URL <https://cutt.ly/mpc-matlab>.
- Rawlings, J., Mayne, D. e Diehl, M. (2017). *Controle preditivo baseado em modelos: teoria, computação e projeto*. Nob Hill Publishing.
- Sopasakis, P., Fresk, E., e Patrinos, P. (2020). OpEn: Geração de código para otimização não convexa incorporada. *IFAC-PapersOnLine*, 53(2), 6548–6554.
- Vanroye, L., Vochten, M., De Schutter, J. e Decr'e, W. (2022). Geração rápida de trajetórias baseada em otimização para aplicações online. In *Workshop on Intelligent Control Methods and Machine Learning Algorithms for Human-Robot Interaction and Assistive Robotics, ICRA 2022*. Philadelphia, USA.
- Verbandt, M., Jacobs, L., Turk, D., Singh, T., Swevers, J., e Pipeleers, G. (2018). Caixa de ferramentas de controle linear - suporte a b-splines no controle lqv. *Mechatronics (Oxford)*, 52, 78–89.
- Verschueren, R., Frison, G., Kouzoupis, D., Frey, J., van Duijkeren, N., Zanelli, A., Novoselnic, B., Albin, T., Quirynen, R., e Diehl, M. (2021). acados — uma estrutura modular de código aberto para controle ideal integrado e rápido. *Computação de Programação Matemática*, 14(1), 147–183.
- W'achter, A. e Biegler, LT (2006). Sobre a implementação de um algoritmo de busca linear de filtro de ponto interior para programação não linear de grande escala. *Programação Matemática*, 106(1), 25–57.